



به نام خدا

مصطفی چرکزی

آزمایش نهم پردازش سیگنال های دیجیتال

آزمایش ۹: پردازش سیگنال‌های صوتی

خواندن فایل استریوی صوتی، پخش آن و تبدیل آن به مونو، و انتخاب یک بخش از فایل صوتی

الف) با دستور `[y Fs] = audioread('Homayoun.mp3');` ابتدا فایل صوتی را بخوانید.

سایز `y` را با دستور `size(y)` مشاهده کنید، آیا میدانید هر عدد یعنی چه؟

```
file_url = "files/Homayoun.mp3";  
[y, fs] = audioread(file_url);  
  
[num_samples, num_channels] = size(y);  
fprintf("Number of samples : %d\n", num_samples);  
fprintf("Number of channels : %d\n", num_channels);
```

Command Window

```
Number of samples : 6764016  
Number of channels : 2
```

ب) می‌خواهیم فایل خوانده شده را در MATLAB پخش کنیم، برای این منظور از دو دستور زیر استفاده می‌کنیم

```
p = audioplayer(y.Fs);
```

```
play(p);
```

```
p = audioplayer(y, fs);  
play(p);
```

ج) برای مونو کردن فایل، چند راه داریم، فقط صدای روی کانال ۱ را برداریم، یا فقط کانال ۲ را یا نمونه های دیجیتال دو کانال را با هم جمع و بر ۲ تقسیم کنیم. که از راه سوم استفاده میکنیم.

```
ymono = sum(y, 2)/2;

[num_samples_ymono, num_channels_ymono] = size(ymono);
fprintf("Number of samples : %d\n", num_samples_ymono);
fprintf("Number of channels : %d\n", num_channels_ymono);

% pmono = audioplayer(ymono, fs);
% play(pmono);
```

د) میخواهیم حدود ۱۰ ثانیه از فایل صوتی را از ثانیه ۳۱ تا ثانیه ۴۱ جدا کنیم و در یک فایل جدید ذخیره کنیم.

با توجه به اینکه فرکانس نمونه برداری را داریم $F_s=44100$ ، لذا میدانیم که ثانیه ۳۱، یعنی سَمپل شماره 31×44100 ، لذا زمان شروع و پایان را بصورت زیر تعیین میکنیم.

```
t_start = 31*Fs;
```

```
t_stop = 41*Fs;
```

حال با دستور زیر بازه شروع و پایان را از سیگنال ymono جدا میکنیم و در فایل ynew میریزیم و سپس آن را پخش میکنیم.

```
ynew = ymono(t_start : t_stop);
```

```
p = audioplayer(ynew,Fs);
```

```
play(p);
```

سپس با دستور audiowrite فایل جدا شده را در کامپیوتر و فولدر کاری با فرمت wav. ذخیره سازی میکنیم.

```
audiowrite('homayoon.wav', ynew, Fs)
```

```
% Seperating from 31s to 41s (we have fs, so this is easy)
```

```
sample_min = fs * 31;
```

```
sample_max = fs * 41;
```

```
y_31_41 = ymono(sample_min: sample_max);
```

```
% p_31_41 = audioplayer(y_31_41, fs);
```

```
% play(p_31_41);
```

```
url_31_41 = "files/new_31_41.wav";
```

```
audiowrite(url_31_41, y_31_41, fs);
```

ه) می‌خواهیم صدای پخش صدرا بالا ببریم، برای این منظور یک بهره تعریف میکنیم $A = 2$ و سپس از دستور زیر استفاده میکنیم

```
Amplifiedy = A * ynew;  
p = audioplayer(Amplifiedy,Fs);  
play(p);
```

بهره را زیاد کنید، چه اتفاقی می‌افتد؟ توضیح دهید چرا این اتفاق می‌افتد؟

```
ynew = y_31_41;  
A = 2;  
Amplifiedy = A * ynew;  
p = audioplayer(Amplifiedy, fs);  
play(p);
```

با ضرب شدن بهره A در سیگنال صوتی، صدای بلند تری به گوش ما میرسد چون دامنه سیگنال صوتی افزایش یافته است. اگر بهره بین صفر تا یک باشد ولوم صدا کمتر میشود.

و) در هنگام پخش فرکانس را بجای F_s برابر $2*F_s$ قرار دهید. چه اتفاقی می‌افتد؟ توضیح دهید.

```
p = audioplayer(ynew, 2*Fs);  
play(p);
```

فرکانس را $F_s/2$ قرار دهید. نتیجه را توضیح دهید.

```
% 1. F_1  
p = audioplayer(ynew, 2*fs);  
play(p);  
  
% 1. F_2  
p = audioplayer(ynew, fs/2);  
play(p);
```

وقتی فرکانس پخش را دو برابر میکنیم سرعت پخش هم دوبرابر میشود. همچنین روی زیر و بم بودن صدا نیز تاثیر گذار است. فرکانس پخش را بیشتر کنیم صدا زیر تر و فرکانس پخش کمتر صدا بم تر خواهد شد.

۳- خواندن صدا، ضبط صدا با یک فرکانس نمونه برداری دیگر، پنهان کردن صدا و سپس بازیابی آن

در این تمرین می خواهیم ابتدا یک صدا از خودتان در مورد درس DSP ضبط کرده و سپس آن را در فایل صوتی ایجاد شده در مرحله قبل پنهان کرده و بعنوان پاسخ ارسال کنید.

الف) برای ضبط صدا از دستور `audiorecorder` استفاده میشود میتوانید به `help` متلب مراجعه کنید، اگر از این دستور به صورت `recObj = audiorecorder` استفاده شود، از مقادیر پیش فرض یعنی $F_s = 8000$ ، تعداد بیت ها 8 بیت و تعداد کانالها ۱ یعنی مونو یک شیء `recorder` درست میشود، که فعلا برای ما کفایت میکند.

سپس با استفاده از دستور `recordblocking(recObj, recDuration)` به میزان مقدار ثانیه ای که در متغیر `recDuration` تعیین کرده ایم، صدا ضبط میشود.

برای نشان دادن لحظه شروع به ضبط و لحظه پایان ضبط میتوانیم از دستور `display` بران اعلان روی صفحه استفاده کنیم.

حداکثر ۱۰ ثانیه صدا در از خودتان در مورد درس DSP ضبط کنید. در کد زیر زمان ضبط ۵ ثانیه تعیین شده.

```
recDuration = 5; % record for 5 seconds
disp('Start speaking.');
```

```
recordblocking(recObj, recDuration);
```

```
disp('End of recording. Playing back ...');
```

در مرحله بعد باید صدا را در یک متغیر ریخته و سپس آن را ذخیره کنیم. برای گرفتن صدا و ریختن در یک متغیر از دستور `getaudiodata(recObj)` استفاده میکنیم.

```
my_voice = getaudiodata(recObj);
```

Matlab help:

Description

Use an `audiorecorder` object to record audio data from an input device such as a microphone for processing in MATLAB®. The `audiorecorder` object contains properties that enable additional flexibility during recording. For example, you can pause, resume, or define callbacks using the `audiorecorder` object functions.

Creation

Syntax

```
recorder = audiorecorder
recorder = audiorecorder(Fs,nBits,nChannels)
recorder = audiorecorder(Fs,nBits,nChannels,ID)
```

Description

`recorder = audiorecorder` creates and returns an `audiorecorder` object with these properties:

- `SampleRate`: 8000
- `BitsPerSample`: 8
- `NumChannels`: 1

```

clc; clear; close all;

recObj = audiorecorder;
recDuration = 10;
disp('start speaking. ');
recordblocking(recObj, recDuration);
disp('End of recording. Playing back ...');

my_voice = getaudiodata(recObj);

```

(ب) حال صدای ضبط شده را پخش کنید. در تمرین قبل توضیح داده شده است. دقت کنید که $F_s=8000$ است.

```

Fs = 8000;
p = audioplayer(my_voice, fs);
play(p);

```

```

%% 3. B
Fs = 8000;
p = audioplayer(my_voice, Fs);
play(p);

```

(ج) صدا را با نام My_DSP_voice و با فرمت wav ذخیره کنید. در تمرین قبل در مورد ذخیره یک صدا توضیح داده شده است.

دقت کنید که $F_s=8000$ است.

```

my_voice_url = "files/My_DSP_voice.wav";
audiowrite(my_voice_url, my_voice, Fs);

```

(د) در مرحله بعد می‌خواهیم این صدای ضبط شده را در داخل صدای ۱۰ ثانیه‌ای ذخیره شده در تمرین دوم پنهان کنیم. ساده‌ترین کار جمع دو صدا است. برای پنهان شدن بیشتر می‌توانید تضعیف شده صدای ضبط شده را پنهان کنید. دقت کنید چون فرکانس نمونه برداری‌ها متفاوت است اگر ۵ ثانیه صدا با فرکانس ۸ کیلو ضبط کرده باشید، معادل ۱ ثانیه صدا در فرکانس ۴۰ کیلو خواهد بود، یعنی کل سیگنال صدای شما در ثانیه اول سیگنال آهنگ مثال ۲ قرار خواهد گرفت.

یک فایل m.file دیگر با نام S1_Exc3_part2 ایجاد کنید و دو صدای ذخیره شده را به همراه فرکانس‌های نمونه بردارشان بخوانید.

```

[y1, Fs1] = audioread('homayoon.wav');
[y2, Fs2] = audioread('My_DSP_voice.wav');

```

برای جمع دو بردار باید طول دو بردار یکسان باشند. در حالی که با استفاده از دستور `length(y1)` یا `numel(y2)` یا دستور `size(y1)` میتوانید طول دو سیگنال خوانده شده را مقایسه کنید، متوجه میشوید که طولشان با هم برابر نیست، پس نمیتوانید آنها را جمع کنید.

```
N1 = numel(y1)
N2 = length(y2)
```

بنابراین $N2$ درایه اول بردار $y1$ را با $y2$ جمع میکنیم و در بردار sy میریزیم.

```
sy = y(1:N2)+y2;
```

یک متغیر y تعریف میکنیم و سیگنال با طول بزرگتر را در آن میریزیم. سپس $N2$ مولفه اول آن را با sy جایگزین میکنیم.

```
y = y1;
y(1:N2) = sy;
```

البته میتوان هر دو مرحله را در یک خط بصورت $y(1:N2)=y(1:N2)+y2$ نوشت که دیگر نیازی به تعریف متغیر sy نباشد.

صدای حاصله را با نام `watermarked_voice` ذخیر کنید.

```
audiowrite('watermarked_voice.wav', y, Fs1);
```

```
clc; clear; close all;
```

```
[y1, Fs1] = audioread('files/new_31_41.wav'); N1 = numel(y1);
[y2, Fs2] = audioread('files/My_DSP_voice.wav'); N2 = length(y2);
```

```
sy = y1(1:N2) + y2;
y = y1;
y(1:N2) = sy;
```

```
% p = audioplayer(y, Fs2);
% play(p);
audiowrite('files/watermarked_voice.wav', y, Fs1);
audiowrite('files/watermarked_voice_higher_fs.wav', y, Fs1);
audiowrite('files/watermarked_voice_lower_fs.wav', y, Fs2);
```

ه) حال می‌خواهیم برنامه ای برای استاد بنویسیم که صدای ما را استخراج کند. کلید همان ۱۰ ثانیه صدای آهنگ است. m فایل جدیدی به نام S1_Exc3_part3 ایجاد کنید.

ابتدا دو فایل واترمارک شده و کلید را بخوانید.

```
[y_key,Fs1] = audioread('homayoon.wav');  
[y_watermarked,Fs2] = audioread('watermarked_voice.wav');
```

سپس آنها را از هم کم کنید

```
y = y_watermarked - y_key;
```

و نتیجه را play کنید.

برای play شدن صحیح صدای استخراج شده باید چه کاری انجام دهید؟

```
clc; clear; close all;  
[y_key, Fs1] = audioread('files/new_31_41.wav');  
[y_watermarked, Fs2] = audioread('files/watermarked_voice.wav');  
  
y = y_watermarked - y_key;  
  
fs = 8000;  
p = audioplayer(y, fs);  
play(p);
```

برای پخش شدن صحیح صدای استخراج شده لازم است فرکانس پخش را روی 8000 تنظیم کنیم چون زمان ریکورد صدا هم با همین فرکانس ریکورد کرده بودیم.

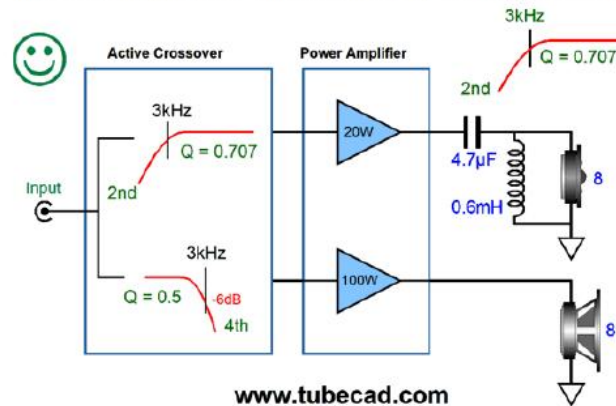
۴- سیستم crossover صوتی دیجیتال

همانطور که میدانید فرکانس شنوایی انسان 20Hz-20KHz است. اما اسپیکرها بدلیل محدودیت های مکانیکی قابلیت پوشش همه این محدوده را ندارند. لذا از دو بلندگو یکی برای پوشش صداهای بم و یکی برای پوشش صداهای زیر استفاده میشود. اما سیستم پردازش صدا باید ابتدا محدوده این دو فرکانس را جدا کند و سپس به بلندگوهای مربوطه ارسال کند، به این سیستم، سیستم crossover میگویند. در واقع

سیستم crossover: سیگنال صوتی را به باندهای فرکانسی مختلف میشکند و هر محدوده را برای اسپیکر مربوطه ارسال میکند. در این تمرین میخواهیم بخش پردازش سیگنال این سیستم را پیاده سازی کنیم.



Improved Bi-Amp Setup with Linkwitz-Riley 4th-Order Crossover



برای انجام این تمرین مراحل زیر را انجام دهید:

الف) با استفاده از دستور `[y,Fs] = audioread('filename.format')` صدای مد نظر را بخوانید. به جای filename.format نام فایل صوتی مدنظرتان را قرار دهید. که در اینصورت sample های فایل در y و فرکانس نمونه برداری در Fs قرار میگیرد.

```
audio_url = "files/My_DSP_voice.wav";  
[y, fs] = audioread(audio_url);
```

ب) با استفاده از دستور $Xf = \text{fft}(y)$ سیگنال را به حوزه فرکانس برده و در متغیر Xf قرار دهید.

```
Xf = fft(y);
```

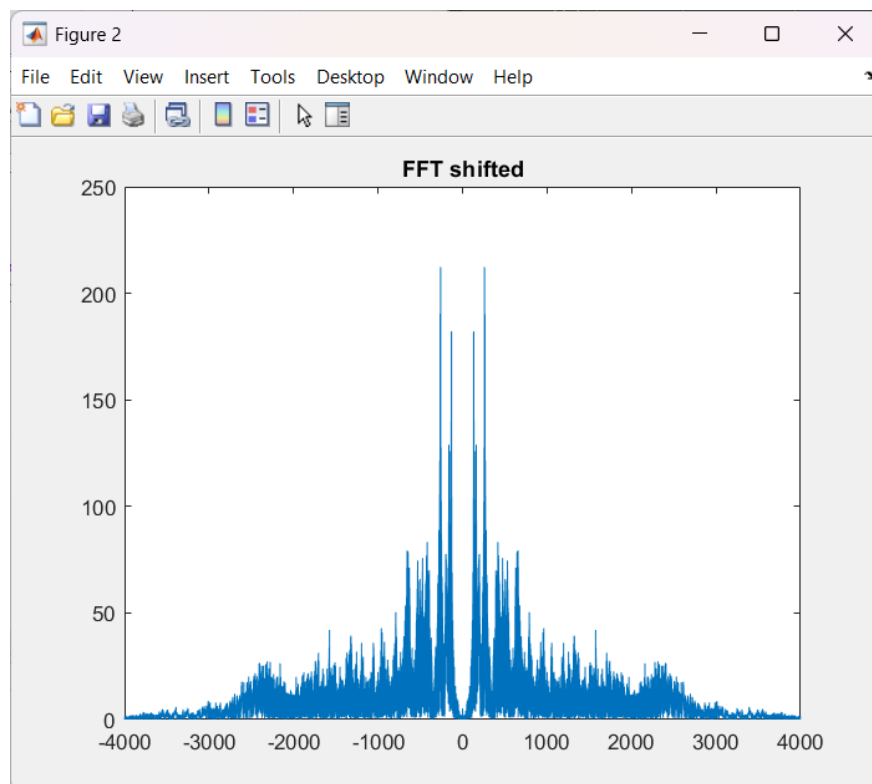
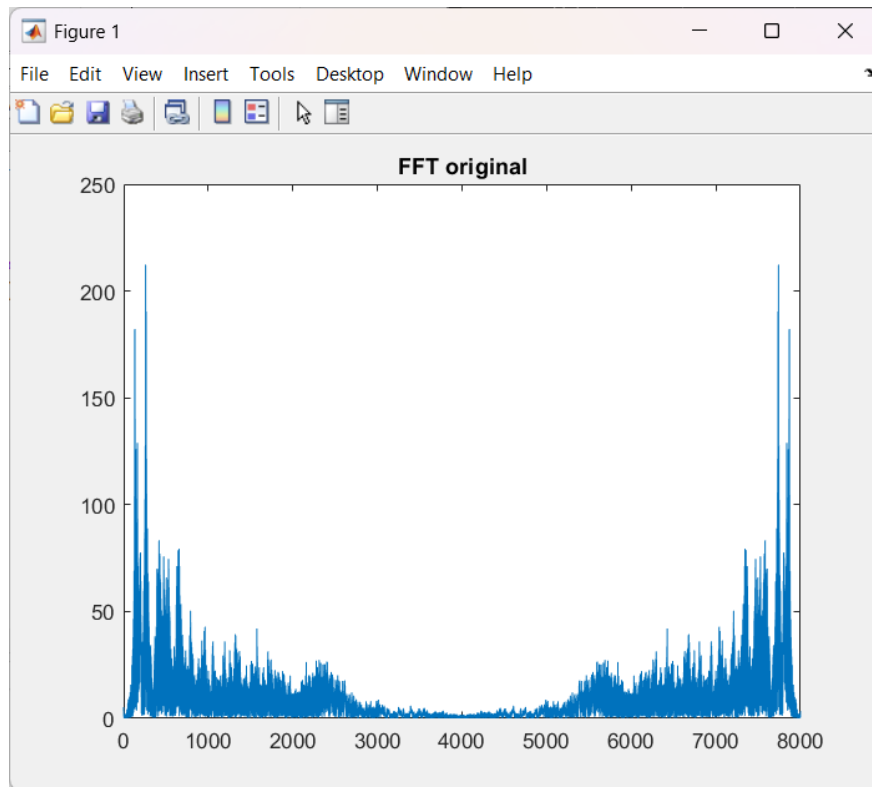
ج) میدانید که دستور fft تبدیل فوریه سیگنال را در بازه $[0 \ 2\pi]$ نشان میدهد و ما علاقه مندیم تبدیل فوریه را در بازه $[-\pi \ \pi]$ مشاهده کنیم، لذا از دستور $\text{fftshift}(Xf)$ برای این منظور استفاده کنید و مقدار آن را در متغیر با نام دلخواهتان مثلاً XfS بریزید.

```
XfS = fftshift(Xf);
```

د) حال میخواهیم هم fft سیگنال و هم شیفت خورده آن را ملاحظه کنیم. برای این منظور باید محور x که همان فرکانس است را درست کنیم. میدانیم که تعداد مولفه های فرکانسی به تعداد سمپل ها است. تعداد سمپل ها N را میتوان با اندازه گیری طول y بدست آورد: $N = \text{length}(y)$ ، و میدانیم که رزولوشن فرکانس برابر است با F_s/N که با f_0 نمایش میدهیم. بنابراین محور فرکانس $f = 0 : f_0 : (N-1)f_0$ خواهد بود. و برای نمایش شیفت یافته محور فرکانس (x) برابر $f = -\left(\frac{N}{2}\right)f_0 : f_0 : \left(\frac{N-1}{2}\right)f_0$ خواهد بود. برای ترسیم در هر حالت نیز میتوانید از دستور plot مثلاً به صورت $\text{plot}(f, \text{abs}(Xf))$ استفاده کنید. دلیل استفاده از abs این است که Xf یک عدد مختلط است یعنی حاوی دامنه و فرکانس است که فعلاً ما با فاز آن کاری نداریم و میخواهیم دامنه را ترسیم کنیم. نکته بعدی اینکه اگر چند plot پشت سر هم استفاده کنید، plot قبلی پاک میشود و آخرین plot نمایش داده میشود. برای اینکه چنین اتفاقی نیافتد و هر تصویر بصورت جداگانه ترسیم شود قبل از plot از یک دستور figure استفاده کنید تا plot جدید در یک figure با شماره جدید ترسیم شود.

```
audio_url = "files/My_DSP_voice.wav";  
[y, fs] = audioread(audio_url);  
  
% 4. b  
Xf = fft(y);  
  
% 4. c  
XfS = fftshift(Xf);  
  
% 4. d  
N = length(Xf);  
f0 = fs/N;  
f = 0: f0 : (N-1) * f0;  
f_shifted = -N/2 * f0: f0 : (N-1)/2 * f0;
```

```
figure;  
plot(f, abs(Xf)); title("FFT original");  
figure;  
plot(f_shifted, abs(XfS)); title("FFT shifted");
```



ه) حال اگر بخواهید محدوده فرکانسی را فیلتر کنید باید پاسخ فرکانسی شیف یافته را در فیلتر پایین گذر ضرب کنید. ساده ترین راه برای ایجاد فیلتر پایین گذر دیجیتال ایده آل، استفاده از دستور `trapmf` است. (این دستور دودنقه تولید میکند) برای این منظور ابتدا فرکانس قطعی که مدنظرتان است را بنویسید مثلا $fc = 3000$ و سپس فیلتر دیجیتال را بصورت زیر تعریف کنید : `LPFilt = trapmf(f,[-fc,-fc,fc,fc])'` (دقت کنید علامت colon در انتها برای ترنسپوز کردن ماتریس الزامی است یعنی بردار ستونی را به سطری تبدیل میکند). فیلتر را با دستور

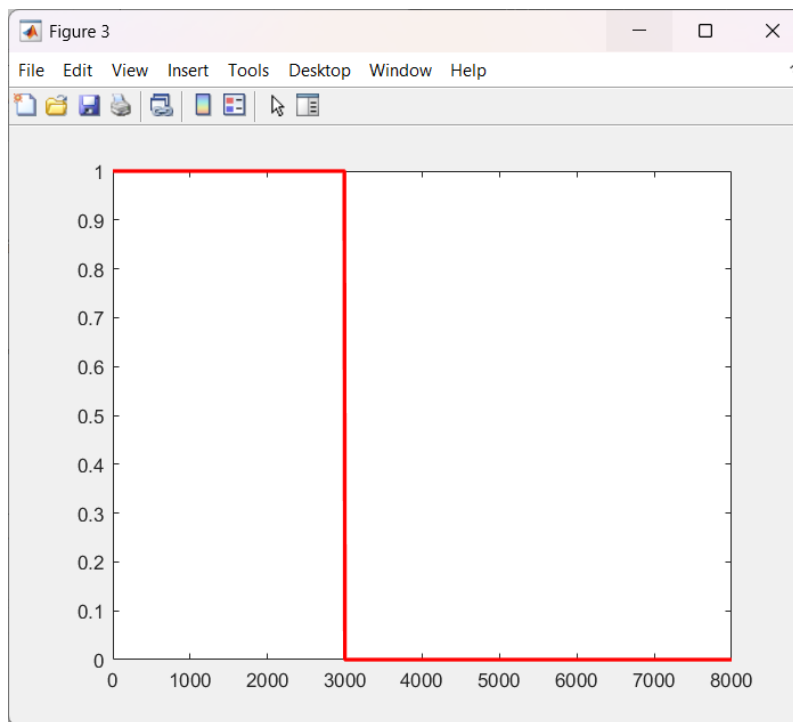
`figure; plot(f,LPFilt,'r')` ترسیم کنید. و نتیجه را ببینید.

حال فیلتر را در سیگنال ضرب نقطه ای کنید. مثلا `XfSFiltered = XfS .* LPFilt`

```
fc = 3000;
LPFilt = trapmf(f, [-fc, -fc, fc, fc])';

figure; plot(f, LPFilt, 'r', 'LineWidth', 2);

XfSFiltered = XfS .* LPFilt;
```



و) برای اینکه اثر اعمال فیلتر را بر روی سیگنال صوتی ببینید، باید سیگنال فیلتر شده را مجدداً به حوزه زمان ببرید. بنابراین ابتدا از دستور `fftshift` باید استفاده کنید و سپس از رابطه `ifft` عکس تبدیل فوریه را محاسبه کنید.

برای مثال اگر نام متغیرهایتان تا اینجا با متن یکی باشد `LPFilterdSignal = ifft(fftshift(XfSFiltered))`

سپس با استفاده از دو دستور زیر میتوانید سیگنال صوتی را پخش کنید.

```
p = audioplayer(real(LPFilterdSignal),Fs);
```

```
play(p);
```

دقت کنید چون ما در سیگنال تغییراتی ایجاد کرده ایم، عکس تبدیل فوریه سیگنال یک عدد مختلط است که ما فقط به قسمت حقیقی آن نیاز داریم لذا برای پخش سیگنال صوتی از دستور `real(LPFilterdSignal)` استفاده کرده ایم. میتوانید مقدار `fc` را در برنامه تغییر دهید و نتیجه را گوش کنید.

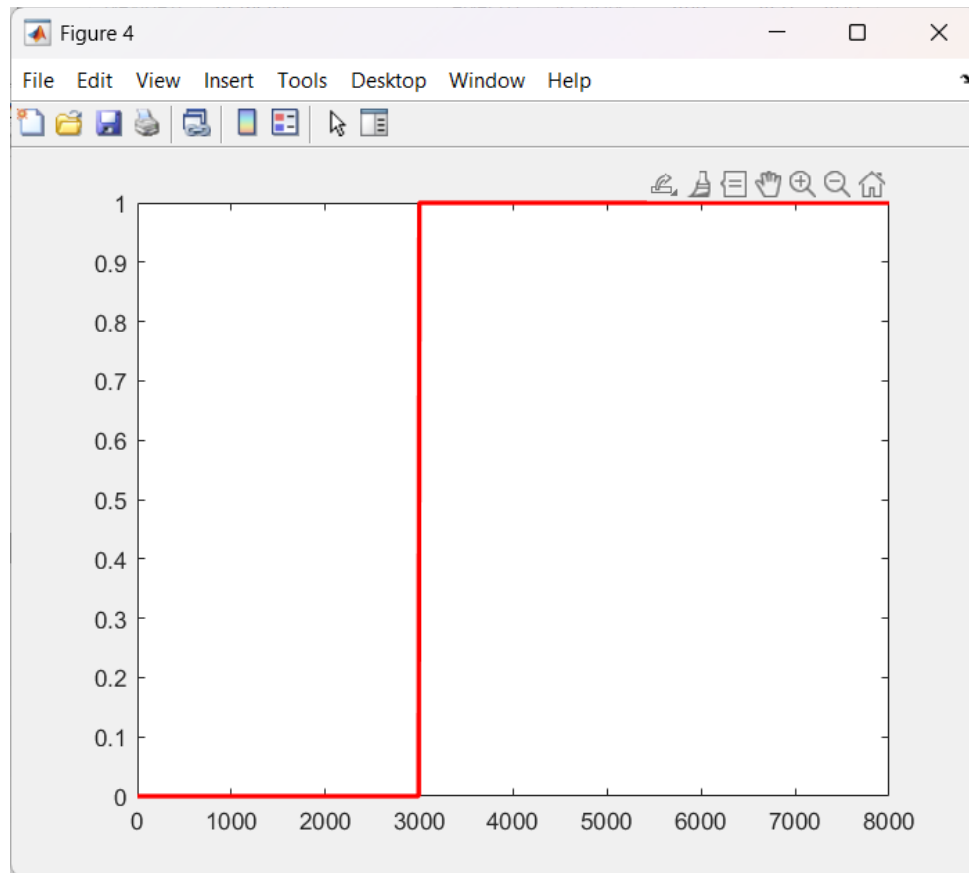
% 4. g

```
LPFilteredSignal = ifft(fftshift(XfSFiltered));  
p = audioplayer(real(LPFilteredSignal), fs);  
play(p);
```

ز) برای اعمال فیلتر بالاگذر ابتدا باید آن را بسازید، ساده ترین راه تفریق فیلتر پایین گذر ساخته شده در مرحله ه از عدد ۱ است. `HPFilt = 1-LPFilt` را نوشته و سپس با دستور `figure; plot(f,HPFilt,'r')` پاسخ فرکانسی فیلتر جدید را ترسیم کنید. حال مجدد فیلتر جدید را در تبدیل فوریه شیف خورده ضرب کنید. سپس عکس تبدیل فوریه را مشابه مرحله واو محاسبه کنید نام آن را `HPFilterdSignal` و سیگنال حاصله را پخش کنید. با فرکانس قطع بازی کنید و نتیجه را گوش کنید.

توجه: در هنگام اجرای برنامه فقط آخرین صدا پخش میشود، یا از بولت باید استفاده کنید، و یا از دستور `pause` بعد از دستور `play` قبل بصورت `pause(10)` استفاده کنید تا هم صدای فرمانس بالا و هم فرمانس پایی را بتوانید گوش بدهید.

```
HPFilt = 1- LPFilt;  
figure; plot(f, HPFilt, 'r', 'LineWidth', 2);  
  
pause(10);  
HPFiltered = XfS .* HPFilt;  
HPFilteredSignal = ifft(fftshift(HPFiltered));  
  
p = audioplayer(real(HPFilteredSignal), fs);  
play(p);
```



ح) می‌خواهیم دو صدا را همزمان بصورت استریو پخش کنیم یعنی فرکانس بالا را از بلندگوی راست و فرکانس پایین را از بلندگوی چپ پخش کنیم. پس باید سیگنال را دو کاناله کنیم :

```
Signalsterio = [HPFilterdSignal, LPFilterdSignal];
```

حال این سیگنال را پخش کنید.

```
p = audioplayer(real(Signalsterio),Fs);
```

```
play(p);
```

```
Signalsterio = [LPFilteredSignal, LPFilteredSignal];
```

```
p = audioplayer(real(Signalsterio), fs);  
play(p);
```

بخش اول آزمایش: خواندن فایل استریوی صوتی، پخش آن و تبدیل آن به مونو و انتخاب یک بخش از فایل صوتی

```
%% 1.
% 1. A
file_url = "files/Homayoun.mp3";
[y, fs] = audioread(file_url);

[num_samples, num_channels] = size(y);
fprintf("Number of samples : %d\n", num_samples);
fprintf("Number of channels : %d\n", num_channels);

% 1. B
%%
p = audioplayer(y, fs);
play(p);
%%
% 1. C
ymono = sum(y, 2)/2;

[num_samples_ymono, num_channels_ymono] = size(ymono);
fprintf("Number of samples : %d\n", num_samples_ymono);
fprintf("Number of channels : %d\n", num_channels_ymono);

% pmono = audioplayer(ymono, fs);
% play(pmono);

% 1. D
% Separating from 31s to 41s (we have fs, so this is easy)

sample_min = fs * 31;
sample_max = fs * 41;

y_31_41 = ymono(sample_min: sample_max);

% p_31_41 = audioplayer(y_31_41, fs);
% play(p_31_41);

url_31_41 = "files/new_31_41.wav";
audiowrite(url_31_41, y_31_41, fs);

% 1. E
ynew = y_31_41;
A = 2;
Amplifiedy = A * ynew;

%%
p = audioplayer(Amplifiedy, fs);
play(p);
%%
% 1. F_1
p = audioplayer(ynew, 2*fs);
play(p);
%%
% 1. F_2
p = audioplayer(ynew, fs/2);
play(p);
```

بخش سوم آزمایش: خواندن صدا، ضبط صدا با یک فرکانس نمونه برداری دیگر، پنهان کردن صدا و سپس بازیابی آن

```
%% 3. A
clc; clear; close all;

recObj = audiorecorder;
recDuration = 10;
disp('start speaking. ');
recordblocking(recObj, recDuration);
disp('End of recording. Playing back ...');

my_voice = getaudiodata(recObj);
% 3. B
Fs = 8000;
%%
p = audioplayer(my_voice, Fs);
play(p);
%%
% 3. C
my_voice_url = 'files/My_DSP_voice.wav';
audiowrite(my_voice_url, my_voice, Fs);
```

```
clc; clear; close all;

[y1, Fs1] = audioread('files/new_31_41.wav'); N1 = numel(y1);
[y2, Fs2] = audioread('files/My_DSP_voice.wav'); N2 = length(y2);

sy = y1(1:N2) + y2;
y = y1;
y(1:N2) = sy;

% p = audioplayer(y, Fs2);
% play(p);
audiowrite('files/watermarked_voice.wav', y, Fs1);
audiowrite('files/watermarked_voice_higher_fs.wav', y, Fs1);
audiowrite('files/watermarked_voice_lower_fs.wav', y, Fs2);
```

```
clc; clear; close all;
[y_key, Fs1] = audioread('files/new_31_41.wav');
[y_watermarked, Fs2] = audioread('files/watermarked_voice.wav');

y = y_watermarked - y_key;

fs = 8000;
p = audioplayer(y, fs);
play(p);
```



```
%% 4 Cross over
% 4. a
clc; clear; close all;
audio_url = "files/My_DSP_voice.wav";
[y, fs] = audioread(audio_url);

% 4. b
Xf = fft(y);

% 4. c
XfS = fftshift(Xf);

% 4. d
N = length(Xf);
f0 = fs/N;
f = 0: f0 : (N-1) * f0;
f_shifted = -N/2 * f0: f0 : (N-1)/2 * f0;

figure;
plot(f, abs(Xf)); title("FFT original");
figure;
plot(f_shifted, abs(XfS)); title("FFT shifted");

% 4. e
fc = 3000;
LPFilt = trapmf(f, [-fc, -fc, fc, fc]);

figure; plot(f, LPFilt, 'r', 'LineWidth', 2);

XfSFiltered = XfS .* LPFilt;

% 4. g
LPFilteredSignal = ifft(fftshift(XfSFiltered));
p = audioplayer(real(LPFilteredSignal), fs);
play(p);

% 4. h
HPFilt = 1- LPFilt;
figure; plot(f, HPFilt, 'r', 'LineWidth', 2);

pause(10);
HPFiltered = XfS .* HPFilt;
HPFilteredSignal = ifft(fftshift(HPFiltered));

p = audioplayer(real(HPFilteredSignal), fs);
play(p);

% 4. i
SignalSterio = [HPFilteredSignal, LPFilteredSignal];

pause(10);
p = audioplayer(real(SignalSterio), fs);
play(p);
```